

```
In [34]: import pandas as pd
import numpy as np
import altair as alt
import seaborn as sns
import matplotlib.pyplot as plt

from scipy.stats import pearsonr, spearmanr
```

```
In [35]: stadium = pd.read_csv('data/processed/clean_stadium.csv')
fanbase = pd.read_csv('data/processed/fanbase_clean.csv')
merch = pd.read_csv('data/processed/cleaned_merch.csv')
fan_merch_merged = pd.read_csv('data/processed/merch_fanbase_merged.csv')
```

/var/folders/wp/pxd2hj7s5j1ckh6z7fz4mwkm0000gn/T/ipykernel_1707/4120701562.py:4: DtypeWarning: Columns (2,3,4,6,7,8,10) have mixed types. Specify dtype option on import or set low_memory=False.

```
fan_merch_merged = pd.read_csv('data/processed/merch_fanbase_merged.csv')
```

Calculate Fan Life Time Value by Age, Region, Season Pass demographics

```
In [36]: fan_ltv = fan_merch_merged.groupby('Member_ID').agg({
    'Unit_Price': 'sum',
    'Barcode': 'count',
    'Games_Attended': 'first',
    'Seasonal_Pass': 'first',
    'Customer_Age_Group': 'first',
    'Customer_Region': 'first'
}).rename(columns={
    'Unit_Price': 'Total_Merch_Spend',
    'Barcode': 'Purchase_Count'
}).reset_index()
```

```
In [37]: ticket_sources = ['Upper Bowl', 'Lower Bowl', 'Season', 'Premium']
total_ticket_revenue = stadium[stadium['Source'].isin(ticket_sources)]['Revenue']
total_attendance = fan_merch_merged['Games_Attended'].sum()

avg_ticket_price = total_ticket_revenue / total_attendance

print(f"Estimated average ticket price: ${avg_ticket_price:.2f}")
print(f"Total ticket revenue in dataset: ${total_ticket_revenue:,.2f}")
print(f"Total attendance in dataset: {total_attendance:,}")
```

Estimated average ticket price: \$54.28
 Total ticket revenue in dataset: \$34,654,688.00
 Total attendance in dataset: 638,480

```
In [38]: fan_ltv['Ticket_Revenue'] = fan_ltv['Games_Attended'] * avg_ticket_price

fan_ltv['Merchandise_LTV'] = fan_ltv['Total_Merch_Spend']
fan_ltv['Total_LTV'] = fan_ltv['Ticket_Revenue'] + fan_ltv['Merchandise_LTV']
```

In [39]: fan_ltv

Out[39]:

	Member_ID	Total_Merch_Spend	Purchase_Count	Games_Attended	Seasonal
0	1370	0.0	0	4	
1	7913	0.0	0	7	
2	9041	0.0	0	4	
3	13517	0.0	0	4	
4	18474	0.0	0	4	
...	...	...	...	...	...
69995	99977604	0.0	0	1	
69996	99986747	0.0	0	7	
69997	99991696	0.0	0	1	
69998	99995269	0.0	0	6	
69999	99999245	0.0	0	6	

70000 rows × 10 columns

```
In [40]: # Segment by Age Group
ltv_by_age = fan_ltv.groupby('Customer_Age_Group').agg({
    'Total_LTV': ['mean', 'median', 'sum'],
    'Ticket_Revenue': 'mean',
    'Merchandise_LTV': 'mean',
    'Member_ID': 'count'
}).round(2)

# Segment by Region
ltv_by_region = fan_ltv.groupby('Customer_Region').agg({
    'Total_LTV': ['mean', 'median', 'sum'],
    'Ticket_Revenue': 'mean',
    'Merchandise_LTV': 'mean',
    'Member_ID': 'count'
}).round(2)

# Segment by Season Pass Status
ltv_by_pass = fan_ltv.groupby('Seasonal_Pass').agg({
    'Total_LTV': ['mean', 'median', 'sum'],
    'Ticket_Revenue': 'mean',
    'Merchandise_LTV': 'mean',
    'Games_Attended': 'mean',
    'Member_ID': 'count'
}).round(2)
```

In [41]: ltv_by_age

Out [41]:

			Total_LTV	Ticket_Revenue	Merchandise_LTV
	mean	median	sum	mean	mean
Customer_Age_Group					
18-25	431.17	369.11	13481079.06	308.11	123.07
26-40	436.18	369.11	7597904.68	313.73	122.45
41-60	433.31	369.11	4495134.59	312.49	120.82
60+	431.77	369.11	1483996.93	310.12	121.65
<18	426.02	366.38	3196862.11	305.70	120.32

In [42]: ltv_by_pass

Out [42]:

				Total_LTV	Ticket_Revenue	Merchandise_LTV	Ga
				sum	mean	mean	
Seasonal_Pass							
False	365.96	346.38	23879018.39		243.86	122.10	
True	1342.59	1338.37	6375958.98		1218.75	123.83	

In [43]: ltv_by_region

Out [43]:

				Total_LTV	Ticket_Revenue	Merchandise_LTV	M
	mean	median		sum	mean	mean	
Customer_Region							
Canada	308.88	271.38		7069940.63	308.88	0.00	
China	315.25	244.25		46026.78	315.25	0.00	
Domestic	507.44	439.94		20690042.73	310.76	196.68	
India	334.03	271.38		80166.92	334.03	0.00	
International	449.00	395.66		1681492.46	305.95	143.05	
Japan	290.02	217.11		48143.57	290.02	0.00	
South Korea	316.62	271.38		39893.49	316.62	0.00	
UK	268.09	271.38		8847.13	268.09	0.00	
USA	313.72	271.38		590423.66	313.72	0.00	

We find no correlation between number of games attending and money spent on merchandise

```

In [44]: # Calculate correlation between games attended and merchandise spending
correlation_data = fan_ltv[['Games_Attended', 'Merchandise_LTV']].copy()

# Remove any rows with missing values
correlation_data = correlation_data.dropna()

# Calculate correlation coefficients
pearson_corr, pearson_p = pearsonr(correlation_data['Games_Attended'],
                                   correlation_data['Merchandise_LTV'])
spearman_corr, spearman_p = spearmanr(correlation_data['Games_Attended'],
                                       correlation_data['Merchandise_LTV'])

print(f"Pearson Correlation: {pearson_corr:.3f} (p-value: {pearson_p:.4f})")
print(f"Spearman Correlation: {spearman_corr:.3f} (p-value: {spearman_p:.4f})")

# Create attendance bins
fan_ltv['Attendance_Bin'] = pd.cut(fan_ltv['Games_Attended'],
                                   bins=[0, 3, 8, 13, 20],
                                   labels=['Low (0-3)', 'Medium (4-8)',
                                           'High (9-13)', 'Very High (14+)'])

# Compare merchandise spending across attendance bins
merch_by_attendance = fan_ltv.groupby('Attendance_Bin').agg({
    'Merchandise_LTV': ['mean', 'median', 'sum'],
    'Member_ID': 'count',
    'Games_Attended': 'mean'
}).round(2)

merch_by_attendance

```

Pearson Correlation: 0.002 (p-value: 0.6218)
Spearman Correlation: -0.004 (p-value: 0.3353)

/var/folders/wp/pxd2hj7s5j1ckh6z7fz4mwkm0000gn/T/ipykernel_1707/2150300712.py:23: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence the warning.

```
merch_by_attendance = fan_ltv.groupby('Attendance_Bin').agg({
```

Out[44]:

	Merchandise_LTV		Member_ID	Games_Attended
	mean	median	sum	count
Attendance_Bin				
Low (0-3)	122.68	75.0	3014890.0	24576
Medium (4-8)	121.75	75.0	4952162.0	40675
High (9-13)	NaN	NaN	0.0	0
Very High (14+)	121.05	75.0	219338.0	1812

	mean	median	sum	count	mean
Attendance_Bin					
Low (0-3)	122.68	75.0	3014890.0	24576	2.00
Medium (4-8)	121.75	75.0	4952162.0	40675	6.00
High (9-13)	NaN	NaN	0.0	0	NaN
Very High (14+)	121.05	75.0	219338.0	1812	17.59

```

In [45]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

```

```

from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import seaborn as sns

# Create feature set for clustering
clustering_features = fan_ltv[['Games_Attended', 'Merchandise_LTV', 'Seasonal_Pass']]

# Add additional behavioral features
# Purchase frequency
purchase_freq = fan_merch_merged.groupby('Member_ID').agg({
    'Barcode': 'count', # Number of transactions
    'Item_Category': 'nunique', # Variety of items purchased
    'Promotion': 'mean', # Proportion of promotional purchases
    'Channel': lambda x: (x == 'Online').sum() / len(x) # Online purchase rate
}).rename(columns={
    'Barcode': 'Purchase_Count',
    'Item_Category': 'Item_Variety',
    'Promotion': 'Promo_Rate',
    'Channel': 'Online_Rate'
}).reset_index()

```

In [46]: clustering_features

Out[46]:

	Games_Attended	Merchandise_LTV	Seasonal_Pass	Member_ID
0	4	0.0	False	1370
1	7	0.0	False	7913
2	4	0.0	False	9041
3	4	0.0	False	13517
4	4	0.0	False	18474
...	...	...	...	...
69995	1	0.0	False	99977604
69996	7	0.0	False	99986747
69997	1	0.0	False	99991696
69998	6	0.0	False	99995269
69999	6	0.0	False	99999245

70000 rows × 4 columns

```

In [47]: # Merge back to clustering dataset
clustering_data = clustering_features.merge(purchase_freq, on='Member_ID', how='left')
clustering_data = clustering_data.fillna(0) # Fill NaN for members with no purchase

# Convert boolean to numeric
clustering_data['Seasonal_Pass'] = clustering_data['Seasonal_Pass'].astype(int)

```

```
print("Clustering Features:")
print(clustering_data.describe())
```

Clustering Features:

	Games_Attended	Merchandise_LTV	Seasonal_Pass	Member_ID \
count	70000.000000	70000.000000	70000.000000	7.000000e+04
mean	5.711414	122.216343	0.067843	2.464124e+07
std	5.168150	156.143214	0.251478	2.595011e+07
min	1.000000	0.000000	0.000000	1.370000e+03
25%	3.000000	0.000000	0.000000	1.001510e+07
50%	5.000000	75.000000	0.000000	1.003260e+07
75%	7.000000	152.000000	0.000000	3.167822e+07
max	30.000000	608.000000	1.000000	9.999924e+07

	Purchase_Count	Item_Variety	Promo_Rate	Online_Rate
count	70000.000000	70000.000000	70000.000000	70000.000000
mean	1.229271	0.889743	0.228907	0.509036
std	1.297531	0.778236	0.372434	0.474496
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	0.000000	0.000000	0.000000
50%	1.000000	1.000000	0.000000	0.500000
75%	2.000000	2.000000	0.500000	1.000000
max	4.000000	2.000000	1.000000	1.000000

```
/var/folders/wp/pxd2hj7s5jlckh6z7fz4mwkm0000gn/T/ipykernel_1707/3923265590.py:3: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill, .bfill is deprecated and will change in a future version. Call result.infer_objects(copy=False) instead. To opt-in to the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
clustering_data = clustering_data.fillna(0) # Fill NaN for members with no purchases
```

```
In [48]: from sklearn.metrics import silhouette_score, davies_bouldin_score
import random
```

```
# Standardize features (important for K-means)
scaler = StandardScaler()
features_scaled = scaler.fit_transform(clustering_data)
```

```
In [49]: random.seed(123)

sample_size = min(10000, len(features_scaled))
sample_indices = np.random.choice(len(features_scaled), sample_size, replace=False)
features_sample = features_scaled[sample_indices]

# Run optimization on sample
inertias = []
silhouette_scores = []
K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(features_sample)
    inertias.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(features_sample, kmeans.labels_))

# Then use optimal K on full dataset
```

```

optimal_k = K_range[np.argmax(silhouette_scores)]
final_kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
final_labels = final_kmeans.fit_predict(features_scaled)

```

```

In [50]: # Plot results
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

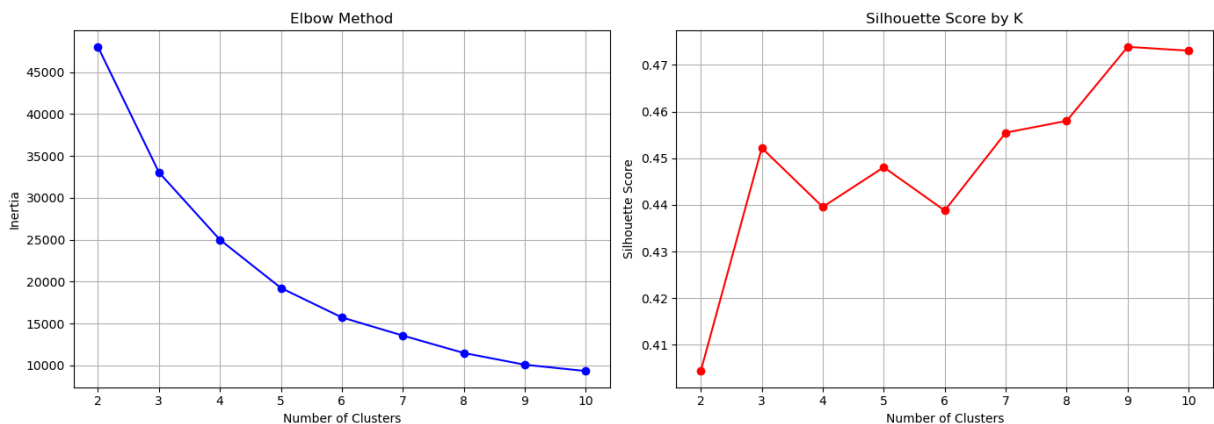
# Elbow plot
axes[0].plot(K_range, inertias, 'bo-')
axes[0].set_xlabel('Number of Clusters')
axes[0].set_ylabel('Inertia')
axes[0].set_title('Elbow Method')
axes[0].grid(True)

# Silhouette score
axes[1].plot(K_range, silhouette_scores, 'ro-')
axes[1].set_xlabel('Number of Clusters')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Score by K')
axes[1].grid(True)

plt.tight_layout()
plt.show()

print("\nSilhouette Scores:")
for k, score in zip(K_range, silhouette_scores):
    print(f"K={k}: {score:.3f}")

```



Silhouette Scores:

```

K=2: 0.404
K=3: 0.452
K=4: 0.440
K=5: 0.448
K=6: 0.439
K=7: 0.455
K=8: 0.458
K=9: 0.474
K=10: 0.473

```

```

In [51]: optimal_k = 4 #anything more than 4 resulted in identical clusters, despite

# Fit final model
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)

```

```
cluster_labels = kmeans.fit_predict(features_scaled)

# Add cluster labels back to original data
fan_ltv['Cluster'] = cluster_labels
clustering_data['Cluster'] = cluster_labels
```

```
In [52]: # Calculate cluster characteristics
cluster_profiles = fan_ltv.groupby('Cluster').agg({
    'Total_LTV': ['mean', 'median', 'sum'],
    'Ticket_Revenue': 'mean',
    'Merchandise_LTV': 'mean',
    'Games_Attended': 'mean',
    'Seasonal_Pass': 'mean', # Proportion with season pass
    'Member_ID': 'count'
}).round(2)

cluster_profiles.columns = ['_'.join(col).strip() for col in cluster_profiles.columns]
print("\nCLUSTER PROFILES")
print(cluster_profiles)

# Add behavioral characteristics
behavior_profiles = clustering_data.groupby('Cluster').agg({
    'Purchase_Count': 'mean',
    'Item_Variety': 'mean',
    'Promo_Rate': 'mean',
    'Online_Rate': 'mean'
}).round(2)

print("\nBEHAVIORAL PROFILES")
print(behavior_profiles)

# Demographic breakdown by cluster
demo_profiles = fan_ltv.groupby(['Cluster', 'Customer_Age_Group']).size().unstack()
print("\nAGE DISTRIBUTION BY CLUSTER")
print(demo_profiles)

region_profiles = fan_ltv.groupby(['Cluster', 'Customer_Region']).size().unstack()
print("\nREGION DISTRIBUTION BY CLUSTER")
print(region_profiles)
```

CLUSTER PROFILES

n \ Cluster	Total_LTV_mean	Total_LTV_median	Total_LTV_sum	Ticket_Revenue_mean
0	365.87	366.38	12077115.92	242.7
1	244.97	271.38	5827977.58	244.9
2	706.89	716.55	5973924.88	245.0
3	1342.59	1338.37	6375958.98	1218.7

Cluster	Merchandise_LTV_mean	Games_Attended_mean	Seasonal_Pass_mean
0	123.13	4.47	0.0
1	0.00	4.51	0.0
2	461.81	4.52	0.0
3	123.83	22.45	1.0

Cluster	Member_ID_count
0	33009
1	23791
2	8451
3	4749

BEHAVIORAL PROFILES

Cluster	Purchase_Count	Item_Variety	Promo_Rate	Online_Rate
0	1.40	1.34	0.34	0.75
1	0.00	0.00	0.00	0.00
2	4.00	1.61	0.45	1.00
3	1.25	0.90	0.23	0.51

AGE DISTRIBUTION BY CLUSTER

Customer_Age_Group	18-25	26-40	41-60	60+	<18
Cluster					
0	14686	8095	4803	1620	3805
1	10694	5949	3589	1178	2381
2	3846	2132	1247	394	832
3	2040	1243	735	245	486

REGION DISTRIBUTION BY CLUSTER

Customer_Region	Canada	China	Domestic	India	International	Japan
Cluster						
0	0	0	29718	0	3291	0
1	21371	135	0	221	0	157
2	0	0	8242	0	209	0
3	1518	11	2813	19	245	9

Customer_Region	South Korea	UK	USA
Cluster			
0	0	0	0
1	119	32	1756

2	0	0	0
3	7	1	126

In [53]: `fan_ltv.head()`

Out[53]:

	Member_ID	Total_Merch_Spend	Purchase_Count	Games_Attended	Seasonal_Pass
0	1370	0.0	0	4	False
1	7913	0.0	0	7	False
2	9041	0.0	0	4	False
3	13517	0.0	0	4	False
4	18474	0.0	0	4	False

The below code for visualizations was written with the help of Microsoft Copilot

```
In [54]: # PCA for visualization
pca = PCA(n_components=2)
features_pca = pca.fit_transform(features_scaled)

plt.figure(figsize=(12, 8))
scatter = plt.scatter(features_pca[:, 0], features_pca[:, 1],
                      c=cluster_labels, cmap='viridis', alpha=0.6, s=50)
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
plt.title('Fan Clusters (PCA Visualization)')
plt.colorbar(scatter, label='Cluster')
plt.grid(True, alpha=0.3)
plt.show()

# Radar chart for cluster comparison
from math import pi

categories = ['Games_Attended', 'Merchandise_LTV', 'Purchase_Count',
             'Item_Variety', 'Seasonal_Pass']

# Normalize values for radar chart (0-1 scale)
cluster_radar = clustering_data.groupby('Cluster')[categories].mean()
cluster_radar_norm = (cluster_radar - cluster_radar.min()) / (cluster_radar.max() - cluster_radar.min())

fig, axes = plt.subplots(1, optimal_k, figsize=(20, 5), subplot_kw=dict(proj='polar'))

for idx, cluster in enumerate(range(optimal_k)):
    ax = axes[idx] if optimal_k > 1 else axes

    values = cluster_radar_norm.iloc[cluster].values.tolist()
    values += values[:1] # Complete the circle

    angles = [n / float(len(categories)) * 2 * pi for n in range(len(categories))]
    angles += angles[:1]

    ax.plot(angles, values, 'o-', linewidth=2)
    ax.fill(angles, values, alpha=0.25)
```

```

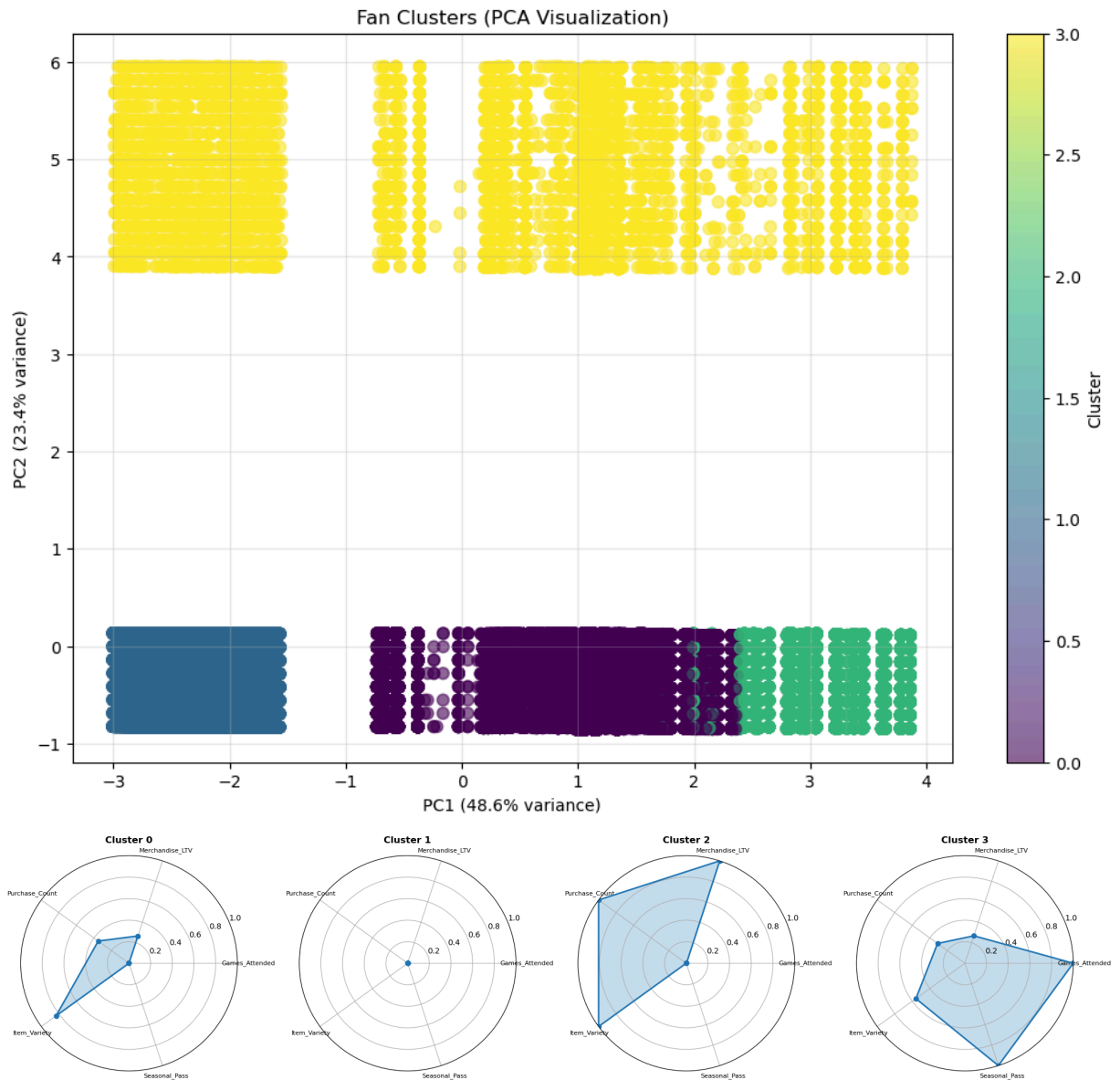
ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, size=8)
ax.set_ylim(0, 1)
ax.set_title(f'Cluster {cluster}', size=12, weight='bold')
ax.grid(True)

```

```

plt.tight_layout()
plt.show()

```



In [55]: *# Create interpretable cluster names based on characteristics*

```

def name_cluster(row):
    """
    Function to assign meaningful names to clusters
    Adjust logic based on your actual cluster profiles
    """
    cluster = row['Cluster']
    games = row['Games_Attended']
    merch = row['Merchandise_LTV']
    season_pass = row['Seasonal_Pass']

```

```

# Example logic - adjust based on your results
if games > 10 and merch > 200 and season_pass == 1:
    return 'Super Fans'
elif games > 10 and season_pass == 1:
    return 'Loyal Attendees'
elif merch > 150:
    return 'Merchandise Enthusiasts'
elif games < 5 and merch < 50:
    return 'Casual Fans'
else:
    return 'Moderate Fans'

for cluster in range(optimal_k):
    cluster_data = fan_ltv[fan_ltv['Cluster'] == cluster]
    print(f"\nCluster {cluster}:")
    print(f"  Size: {len(cluster_data)} fans")
    print(f"  Avg LTV: ${cluster_data['Total_LTV'].mean():.2f}")
    print(f"  Avg Games: {cluster_data['Games_Attended'].mean():.1f}")
    print(f"  Avg Merch: ${cluster_data['Merchandise_LTV'].mean():.2f}")
    print(f"  Season Pass %: {cluster_data['Seasonal_Pass'].mean()*100:.1f}%")

```

Cluster 0:
 Size: 33009 fans
 Avg LTV: \$365.87
 Avg Games: 4.5
 Avg Merch: \$123.13
 Season Pass %: 0.0%

Cluster 1:
 Size: 23791 fans
 Avg LTV: \$244.97
 Avg Games: 4.5
 Avg Merch: \$0.00
 Season Pass %: 0.0%

Cluster 2:
 Size: 8451 fans
 Avg LTV: \$706.89
 Avg Games: 4.5
 Avg Merch: \$461.81
 Season Pass %: 0.0%

Cluster 3:
 Size: 4749 fans
 Avg LTV: \$1342.59
 Avg Games: 22.5
 Avg Merch: \$123.83
 Season Pass %: 100.0%

Cluster 0: Merchandise Enthusiasts

- increase attendance - ticket discounts?

Cluster 1: Disengaged Fans

Cluster 2: Super Fans/VIPs

- exclusive deals and early access (important to retain)

Cluster 3: Loyal Attendees

- upsell merchandise

```
In [56]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

# First, let's verify the cluster assignments make sense
print("="*60)
print("CLUSTER VALIDATION")
print("="*60)

cluster_summary = fan_ltv.groupby('Cluster').agg({
    'Games_Attended': ['mean', 'min', 'max'],
    'Merchandise_LTV': ['mean', 'min', 'max'],
    'Seasonal_Pass': 'mean',
    'Member_ID': 'count'
}).round(2)

print(cluster_summary)

# Check if clusters make sense
print("\n\nCluster characteristics:")
for cluster in sorted(fan_ltv['Cluster'].unique()):
    print(f"\nCluster {cluster}:")
    cluster_data = fan_ltv[fan_ltv['Cluster'] == cluster]
    print(f"  Size: {len(cluster_data)}")
    print(f"  Avg Games: {cluster_data['Games_Attended'].mean():.1f}")
    print(f"  Avg Merch: ${cluster_data['Merchandise_LTV'].mean():.2f}")
    print(f"  Season Pass %: {cluster_data['Seasonal_Pass'].mean()*100:.1f}%")
```

=====							
CLUSTER VALIDATION							
=====							
\	Games_Attended			Merchandise_LTV			Seasonal_Pass
	mean	min	max	mean	min	max	mean
Cluster							
0	4.47	1	8	123.13	20.0	304.0	0.0
1	4.51	1	8	0.00	0.0	0.0	0.0
2	4.52	1	8	461.81	200.0	608.0	0.0
3	22.45	15	30	123.83	0.0	608.0	1.0

Member_ID	
Cluster	count
0	33009
1	23791
2	8451
3	4749

Cluster characteristics:

Cluster 0:
Size: 33009
Avg Games: 4.5
Avg Merch: \$123.13
Season Pass %: 0.0%

Cluster 1:
Size: 23791
Avg Games: 4.5
Avg Merch: \$0.00
Season Pass %: 0.0%

Cluster 2:
Size: 8451
Avg Games: 4.5
Avg Merch: \$461.81
Season Pass %: 0.0%

Cluster 3:
Size: 4749
Avg Games: 22.5
Avg Merch: \$123.83
Season Pass %: 100.0%

```
In [57]: # Prepare clean dataset – one row per member
print("\n" + "="*60)
print("PREPARING CLEAN DATASET")
print("="*60)

# Start fresh with fan_ltv (should already have one row per member)
model_data = fan_ltv[['Member_ID', 'Customer_Age_Group', 'Customer_Region',
                      'Seasonal_Pass', 'Games_Attended', 'Merchandise_LTV',
                      'Cluster']].copy()
```

```

# Remove any duplicates
model_data = model_data.drop_duplicates(subset='Member_ID')
print(f"Total members: {len(model_data)}")

# Check for missing values
print("\nMissing values:")
print(model_data.isnull().sum())

# Remove rows with missing values in key columns
model_data = model_data.dropna(subset=['Customer_Age_Group', 'Customer_Region'])
print(f"After removing NaN: {len(model_data)}")

# Verify cluster distribution
print("\nCluster distribution:")
print(model_data['Cluster'].value_counts().sort_index())

```

```

=====
PREPARING CLEAN DATASET
=====

```

Total members: 70000

Missing values:

Member_ID	0
Customer_Age_Group	0
Customer_Region	0
Seasonal_Pass	0
Games_Attended	0
Merchandise_LTV	0
Cluster	0

dtype: int64

After removing NaN: 70000

Cluster distribution:

Cluster

0 33009

1 23791

2 8451

3 4749

Name: count, dtype: int64

In [58]: *# One-hot encoding is often better than label encoding for tree models*

```

print("\n" + "="*60)
print("ENCODING FEATURES")
print("="*60)

# Create a copy for modeling
modeling_df = model_data.copy()

# One-hot encode categorical variables
age_dummies = pd.get_dummies(modeling_df['Customer_Age_Group'], prefix='Age')
region_dummies = pd.get_dummies(modeling_df['Customer_Region'], prefix='Region')

# Combine all features
feature_df = pd.concat([
    modeling_df,
    age_dummies,
    region_dummies,
])

```

```

    region_dummies,
    modeling_df[['Seasonal_Pass', 'Games_Attended', 'Merchandise_LTV']].astype(
], axis=1)

# Convert boolean to int
if 'Seasonal_Pass' in feature_df.columns:
    feature_df['Seasonal_Pass'] = feature_df['Seasonal_Pass'].astype(int)

print(f"Feature columns: {list(feature_df.columns)}")
print(f"Feature shape: {feature_df.shape}")

# Target variable
y = modeling_df['Cluster'].values

print(f"\nTarget distribution:")
print(pd.Series(y).value_counts().sort_index())

```

=====

ENCODING FEATURES

=====

```

Feature columns: ['Age_18-25', 'Age_26-40', 'Age_41-60', 'Age_60+', 'Age_<18', 'Region_Canada', 'Region_China', 'Region_Domestic', 'Region_India', 'Region_International', 'Region_Japan', 'Region_South Korea', 'Region_UK', 'Region_USA', 'Seasonal_Pass', 'Games_Attended', 'Merchandise_LTV']
Feature shape: (70000, 17)

```

Target distribution:

```

0    33009
1    23791
2     8451
3     4749

```

Name: count, dtype: int64

```

In [59]: print("\n" + "="*60)
print("TRAIN-TEST SPLIT")
print("="*60)

X_train, X_test, y_train, y_test = train_test_split(
    feature_df, y,
    test_size=0.2,
    random_state=42,
    stratify=y # Ensures balanced classes
)

print(f"Training set: {len(X_train)} samples")
print(f"Test set: {len(X_test)} samples")
print(f"\nTraining cluster distribution:")
print(pd.Series(y_train).value_counts().sort_index())
print(f"\nTest cluster distribution:")
print(pd.Series(y_test).value_counts().sort_index())

```

```
=====
TRAIN-TEST SPLIT
=====
```

```
Training set: 56000 samples
Test set: 14000 samples
```

```
Training cluster distribution:
```

```
0    26407
1    19033
2     6761
3     3799
```

```
Name: count, dtype: int64
```

```
Test cluster distribution:
```

```
0     6602
1     4758
2     1690
3       950
```

```
Name: count, dtype: int64
```

```
In [60]: print("\n" + "="*60)
print("TRAINING DECISION TREE")
print("="*60)

# Train decision tree
dt_model = DecisionTreeClassifier(
    max_depth=10,
    min_samples_split=50,
    min_samples_leaf=20,
    random_state=42
)

dt_model.fit(X_train, y_train)

# Predictions
y_train_pred = dt_model.predict(X_train)
y_test_pred = dt_model.predict(X_test)

# Evaluate
train_acc = accuracy_score(y_train, y_train_pred)
test_acc = accuracy_score(y_test, y_test_pred)

print(f"Training Accuracy: {train_acc:.3f}")
print(f"Test Accuracy: {test_acc:.3f}")

print("\nTest Set Classification Report:")
print(classification_report(y_test, y_test_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_test_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Decision Tree')
plt.ylabel('True Cluster')
plt.xlabel('Predicted Cluster')
```

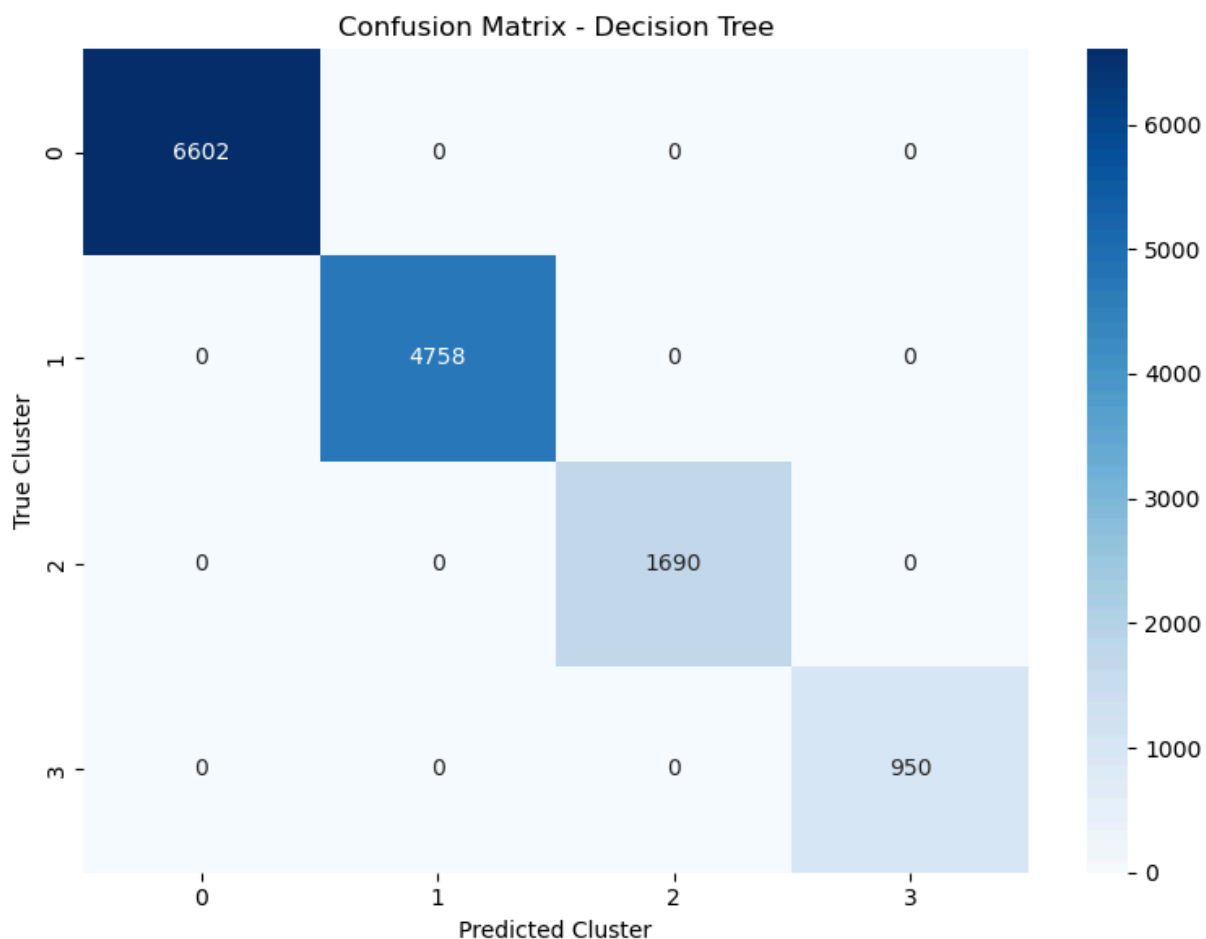
```
plt.tight_layout()
plt.show()
```

TRAINING DECISION TREE

Training Accuracy: 1.000
Test Accuracy: 1.000

Test Set Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6602
1	1.00	1.00	1.00	4758
2	1.00	1.00	1.00	1690
3	1.00	1.00	1.00	950
accuracy			1.00	14000
macro avg	1.00	1.00	1.00	14000
weighted avg	1.00	1.00	1.00	14000



```
In [61]: # Check what the model is learning
feature_importance = pd.DataFrame({
    'Feature': feature_df.columns,
    'Importance': dt_model.feature_importances_
}).sort_values('Importance', ascending=False)
```

```

print("\n" + "="*60)
print("TOP 10 MOST IMPORTANT FEATURES")
print("="*60)
print(feature_importance.head(10))

plt.figure(figsize=(10, 6))
top_features = feature_importance.head(15)
plt.barh(top_features['Feature'], top_features['Importance'])
plt.xlabel('Importance')
plt.title('Top 15 Feature Importances')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

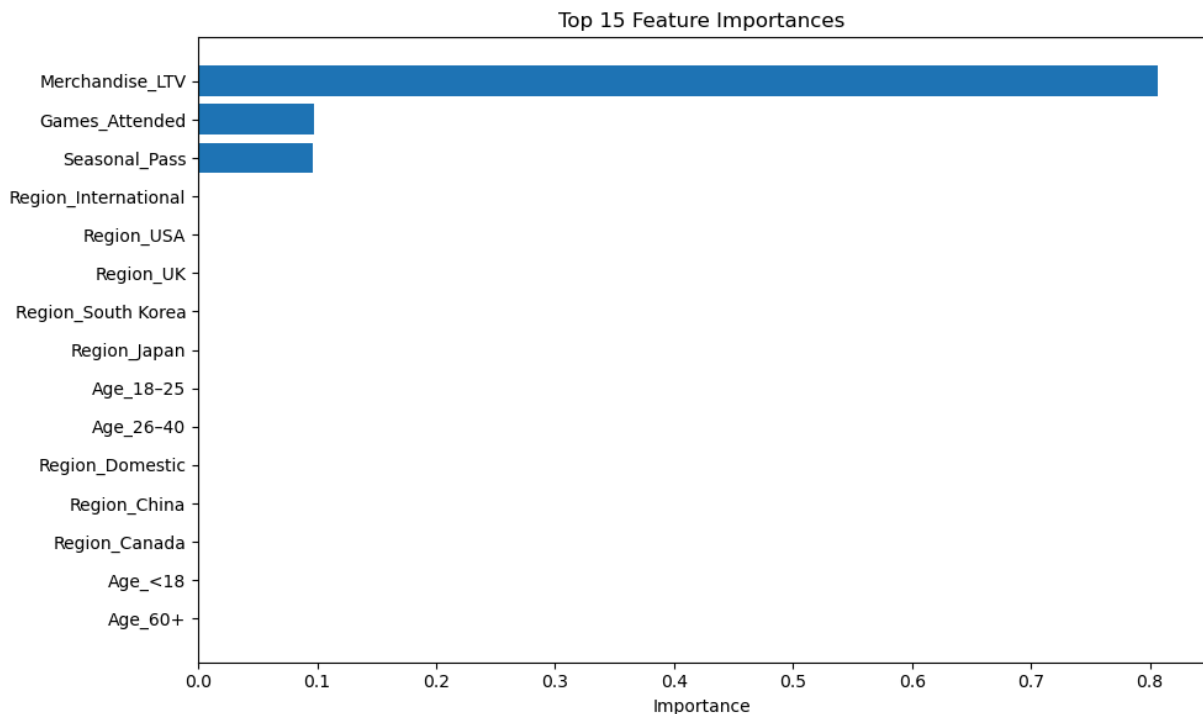
```

```

=====
TOP 10 MOST IMPORTANT FEATURES
=====

```

	Feature	Importance
16	Merchandise_LTV	0.806755
15	Games_Attended	0.096928
14	Seasonal_Pass	0.096318
9	Region_International	0.000000
13	Region_USA	0.000000
12	Region_UK	0.000000
11	Region_South Korea	0.000000
10	Region_Japan	0.000000
0	Age_18-25	0.000000
1	Age_26-40	0.000000



```

In [62]: print("\n" + "="*60)
print("TRAINING RANDOM FOREST")
print("="*60)

rf_model = RandomForestClassifier(
    n_estimators=200,

```

```

    max_depth=15,
    min_samples_split=30,
    min_samples_leaf=10,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

# Predictions
y_train_pred_rf = rf_model.predict(X_train)
y_test_pred_rf = rf_model.predict(X_test)

# Evaluate
train_acc_rf = accuracy_score(y_train, y_train_pred_rf)
test_acc_rf = accuracy_score(y_test, y_test_pred_rf)

print(f"Training Accuracy: {train_acc_rf:.3f}")
print(f"Test Accuracy: {test_acc_rf:.3f}")

print("\nTest Set Classification Report:")
print(classification_report(y_test, y_test_pred_rf))

# Confusion Matrix
cm_rf = confusion_matrix(y_test, y_test_pred_rf)
plt.figure(figsize=(8, 6))
sns.heatmap(cm_rf, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Random Forest')
plt.ylabel('True Cluster')
plt.xlabel('Predicted Cluster')
plt.tight_layout()
plt.show()

```

```

=====
TRAINING RANDOM FOREST
=====

```

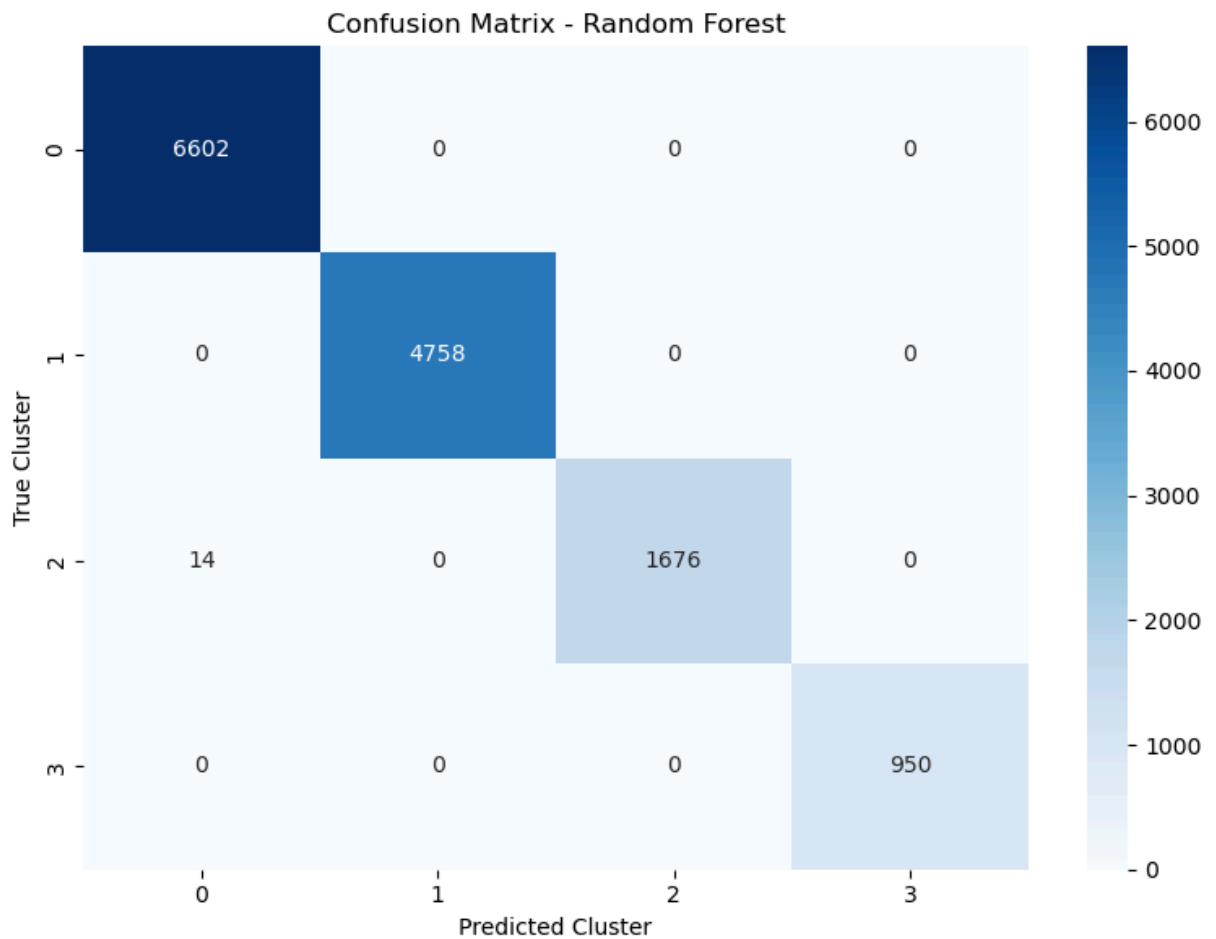
```

Training Accuracy: 0.999
Test Accuracy: 0.999

```

Test Set Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	6602
1	1.00	1.00	1.00	4758
2	1.00	0.99	1.00	1690
3	1.00	1.00	1.00	950
accuracy			1.00	14000
macro avg	1.00	1.00	1.00	14000
weighted avg	1.00	1.00	1.00	14000



```
In [63]: # Compare models
print("\n" + "="*60)
print("MODEL COMPARISON")
print("="*60)
print(f"Decision Tree Test Accuracy: {test_acc:.3f}")
print(f"Random Forest Test Accuracy: {test_acc_rf:.3f}")

# Choose best model
if test_acc_rf > test_acc:
    best_model = rf_model
    best_model_name = "Random Forest"
    print(f"\nUsing Random Forest as final model")
else:
    best_model = dt_model
    best_model_name = "Decision Tree"
    print(f"\nUsing Decision Tree as final model")

# Save everything
joblib.dump(best_model, 'cluster_prediction_model_v2.pkl')
joblib.dump(feature_df.columns.tolist(), 'feature_columns.pkl')
print(f"\nModel saved: cluster_prediction_model_v2.pkl")
print(f"Feature columns saved: feature_columns.pkl")
```

```
=====
MODEL COMPARISON
=====
```

Decision Tree Test Accuracy: 1.000

Random Forest Test Accuracy: 0.999

Using Decision Tree as final model

Model saved: cluster_prediction_model_v2.pkl

Feature columns saved: feature_columns.pkl

```
In [64]: def predict_cluster_new(age_group, region, seasonal_pass, games_attended, merch_ltv):
        """
        Proper prediction function with one-hot encoding
        """
        try:
            # Load model and feature columns
            model = joblib.load('cluster_prediction_model_v2.pkl')
            feature_columns = joblib.load('feature_columns.pkl')

            # Create input dataframe
            input_data = pd.DataFrame({
                'Customer_Age_Group': [age_group],
                'Customer_Region': [region],
                'Seasonal_Pass': [int(seasonal_pass)],
                'Games_Attended': [float(games_attended)],
                'Merchandise_LTV': [float(merch_ltv)]
            })

            # One-hot encode
            age_dummies = pd.get_dummies(input_data['Customer_Age_Group'], prefix='age')
            region_dummies = pd.get_dummies(input_data['Customer_Region'], prefix='region')

            # Combine
            input_features = pd.concat([
                age_dummies,
                region_dummies,
                input_data[['Seasonal_Pass', 'Games_Attended', 'Merchandise_LTV']]
            ], axis=1)

            # Ensure all expected columns exist (add missing ones as 0)
            for col in feature_columns:
                if col not in input_features.columns:
                    input_features[col] = 0

            # Reorder columns to match training data
            input_features = input_features[feature_columns]

            # Predict
            cluster = model.predict(input_features)[0]

            # Get probabilities if available
            if hasattr(model, 'predict_proba'):
                probabilities = model.predict_proba(input_features)[0]
                prob_dict = {"Cluster_{}".format(i): float(prob) for i, prob in enumerate(probabilities)}
            else:
```

```

        prob_dict = {f"Cluster_{cluster}": 1.0}

    # Cluster names (update based on your actual clusters)
    cluster_names = {
        0: "Merchandise Enthusiasts",
        1: "Disengaged/Inactive",
        2: "Super Fans",
        3: "Loyal Attendees"
    }

    return {
        'predicted_cluster': int(cluster),
        'cluster_name': cluster_names.get(cluster, f"Cluster {cluster}"),
        'probabilities': prob_dict,
        'confidence': float(max(prob_dict.values()))
    }

except Exception as e:
    return {"error": str(e)}

```

```

In [65]: print("\n" + "="*60)
print("TESTING NEW PREDICTION FUNCTION")
print("="*60)

# Get actual examples from each cluster for testing
for cluster in sorted(fan_ltv['Cluster'].unique()):
    sample = fan_ltv[fan_ltv['Cluster'] == cluster].iloc[0]

    print(f"\n--- Testing Cluster {cluster} Sample ---")
    print(f"Actual: {sample['Customer_Age_Group']}, {sample['Customer_Region']}
          f"Pass={sample['Seasonal_Pass']}, Games={sample['Games_Attended']}
          f"Merch=${sample['Merchandise_LTV']:.2f}")

    result = predict_cluster_new(
        age_group=sample['Customer_Age_Group'],
        region=sample['Customer_Region'],
        seasonal_pass=sample['Seasonal_Pass'],
        games_attended=sample['Games_Attended'],
        merch_ltv=sample['Merchandise_LTV']
    )

    print(f"Predicted: Cluster {result['predicted_cluster']} - {result['cluster_name']}")
    print(f"Confidence: {result['confidence']:.2%}")
    print(f"Match: {'✓' if result['predicted_cluster'] == cluster else 'x'}")

```

```
=====
TESTING NEW PREDICTION FUNCTION
=====
```

--- Testing Cluster 0 Sample ---

Actual: 41-60, Domestic, Pass=False, Games=5, Merch=\$177.00

Predicted: Cluster 0 - Merchandise Enthusiasts

Confidence: 100.00%

Match: ✓

--- Testing Cluster 1 Sample ---

Actual: 26-40, Canada, Pass=False, Games=4, Merch=\$0.00

Predicted: Cluster 1 - Disengaged/Inactive

Confidence: 100.00%

Match: ✓

--- Testing Cluster 2 Sample ---

Actual: 26-40, Domestic, Pass=False, Games=5, Merch=\$300.00

Predicted: Cluster 2 - Super Fans

Confidence: 100.00%

Match: ✓

--- Testing Cluster 3 Sample ---

Actual: 18-25, Canada, Pass=True, Games=19, Merch=\$0.00

Predicted: Cluster 3 - Loyal Attendees

Confidence: 100.00%

Match: ✓

```
In [66]: # =====
# FINAL CLUSTER PREDICTION SYSTEM
# =====

import pandas as pd
import numpy as np
import joblib

class FanClusterPredictor:
    """
    Production-ready fan cluster predictor
    """
    def __init__(self,
                  model_path='cluster_prediction_model_v2.pkl',
                  features_path='feature_columns.pkl'):
        """Initialize the predictor with saved model and features"""
        self.model = joblib.load(model_path)
        self.feature_columns = joblib.load(features_path)

        # Cluster definitions (update these based on your actual clusters)
        self.cluster_info = {
            0: {
                'name': 'Merchandise Enthusiasts',
                'description': 'High merchandise spending, lower attendance',
                'strategy': 'Drive stadium attendance with exclusive in-store',
            },
            1: {
                'name': 'Disengaged/Inactive',
```

```

        'description': 'Low engagement across all metrics',
        'strategy': 'Re-engagement campaigns, special offers'
    },
    2: {
        'name': 'Super Fans',
        'description': 'High attendance and high merchandise spending',
        'strategy': 'VIP experiences, premium offerings, loyalty rewards'
    },
    3: {
        'name': 'Loyal Attendees',
        'description': 'Frequent game attendance, moderate merch spending',
        'strategy': 'Upsell merchandise at games, bundle deals'
    }
}

```

```

def predict_single(self, age_group, region, seasonal_pass,
                    games_attended=0, merch_ltv=0):
    """
    Predict cluster for a single fan

    Parameters:
    -----
    age_group : str
        Age bracket (e.g., '<18', '18-25', '26-40', '41-60', '60+')
    region : str
        Geographic region (e.g., 'Canada', 'USA', 'Japan', etc.)
    seasonal_pass : bool
        Whether fan has a season pass
    games_attended : int
        Number of games attended (default 0 for new members)
    merch_ltv : float
        Total merchandise spending (default 0 for new members)

    Returns:
    -----
    dict with prediction, probabilities, and recommendations
    """
    try:
        # Create input dataframe
        input_data = pd.DataFrame({
            'Customer_Age_Group': [age_group],
            'Customer_Region': [region],
            'Seasonal_Pass': [int(seasonal_pass)],
            'Games_Attended': [float(games_attended)],
            'Merchandise_LTV': [float(merch_ltv)]
        })

        # One-hot encode
        age_dummies = pd.get_dummies(input_data['Customer_Age_Group'],
                                     prefix='Age', drop_first=False)
        region_dummies = pd.get_dummies(input_data['Customer_Region'],
                                       prefix='Region', drop_first=False)

        # Combine features
        input_features = pd.concat([
            age_dummies,

```

```

        region_dummies,
        input_data[['Seasonal_Pass', 'Games_Attended', 'Merchandise_
], axis=1)

# Ensure all expected columns exist
for col in self.feature_columns:
    if col not in input_features.columns:
        input_features[col] = 0

# Reorder columns to match training data
input_features = input_features[self.feature_columns]

# Predict
cluster = self.model.predict(input_features)[0]

# Get probabilities
if hasattr(self.model, 'predict_proba'):
    probabilities = self.model.predict_proba(input_features)[0]
    prob_dict = {i: float(prob) for i, prob in enumerate(probabi
else:
    prob_dict = {cluster: 1.0}

# Get cluster info
cluster_data = self.cluster_info.get(cluster, {
    'name': f'Cluster {cluster}',
    'description': 'Unknown',
    'strategy': 'Unknown'
})

return {
    'success': True,
    'cluster': int(cluster),
    'cluster_name': cluster_data['name'],
    'description': cluster_data['description'],
    'marketing_strategy': cluster_data['strategy'],
    'confidence': float(max(prob_dict.values())),
    'all_probabilities': prob_dict,
    'inputs': {
        'age_group': age_group,
        'region': region,
        'seasonal_pass': seasonal_pass,
        'games_attended': games_attended,
        'merchandise_ltv': merch_ltv
    }
}

except Exception as e:
    return {
        'success': False,
        'error': str(e),
        'error_type': type(e).__name__
    }

def predict_batch(self, members_df):
    """
    Predict clusters for multiple members

```

Parameters:

members_df : DataFrame
DataFrame with columns: Customer_Age_Group, Customer_Region,
Seasonal_Pass, Games_Attended, Merchandise_LTV

Returns:

DataFrame with predictions added
"""

results = []

```
for idx, row in members_df.iterrows():
    result = self.predict_single(
        age_group=row['Customer_Age_Group'],
        region=row['Customer_Region'],
        seasonal_pass=row['Seasonal_Pass'],
        games_attended=row.get('Games_Attended', 0),
        merch_ltv=row.get('Merchandise_LTV', 0)
    )
```

```
    if result['success']:
        results.append({
            'Predicted_Cluster': result['cluster'],
            'Cluster_Name': result['cluster_name'],
            'Confidence': result['confidence']
        })
```

```
    else:
        results.append({
            'Predicted_Cluster': None,
            'Cluster_Name': 'Error',
            'Confidence': 0.0
        })
```

results_df = pd.DataFrame(results)

return pd.concat([members_df.reset_index(drop=True), results_df], ax

def get_cluster_summary(self):

"""Get summary of all clusters"""

```
return pd.DataFrame([
    {
        'Cluster': k,
        'Name': v['name'],
        'Description': v['description'],
        'Strategy': v['strategy']
    }
    for k, v in self.cluster_info.items()
])
```

```
# =====
# Initialize predictor
# =====
predictor = FanClusterPredictor()

print("="*60)
```

```
print("FAN CLUSTER PREDICTOR - READY")
print("="*60)
print(predictor.get_cluster_summary().to_string(index=False))
```

```
=====
FAN CLUSTER PREDICTOR - READY
=====
```

Cluster	Name	Description	Strategy
0	Merchandise Enthusiasts	High merchandise spending, lower attendance	Drive stadium attendance with exclusive in-store items
1	Disengaged/Inactive	Low engagement across all metrics	Re-engagement campaigns, special offers
2	Super Fans	High attendance and high merchandise spending	VIP experiences, premium offerings, loyalty rewards
3	Loyal Attendees	Frequent game attendance, moderate merch spending	Upsell merchandise at games, bundle deals